

CRUD PROYECTO



Eva Pérez Carmona

2ºDAM

08/01/2026

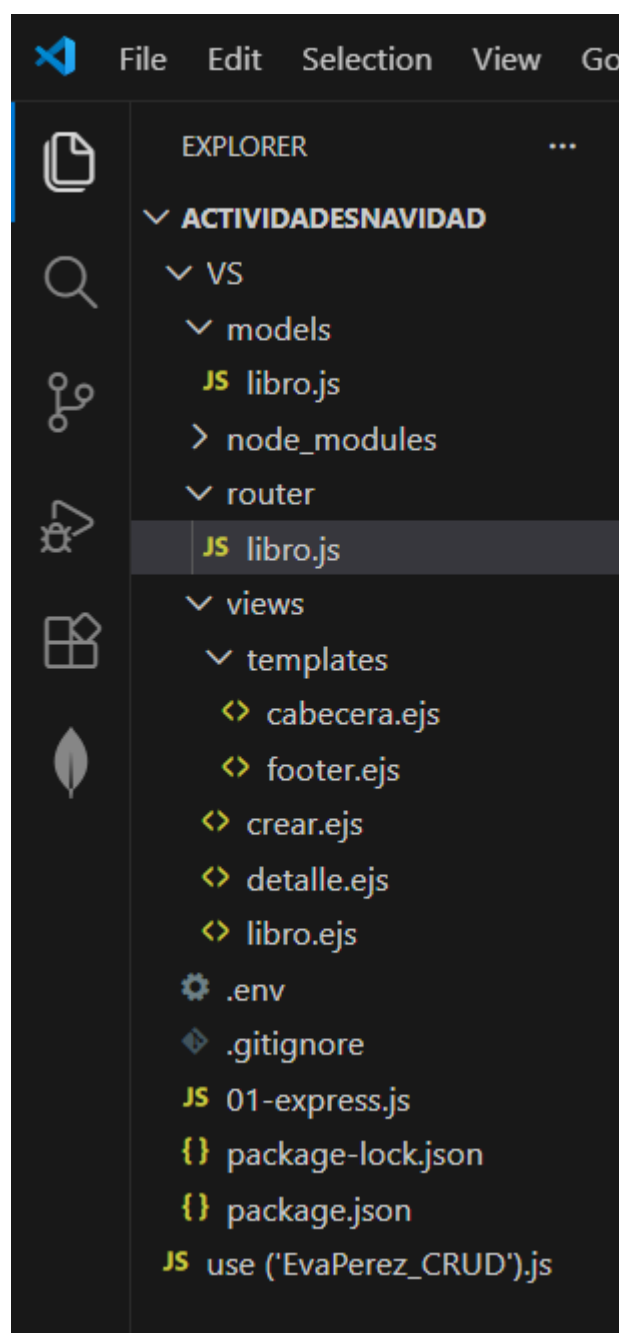
ÍNDICE

CRUD PROYECTO	1
<i>Desarrollo CRUD</i>	<i>3</i>

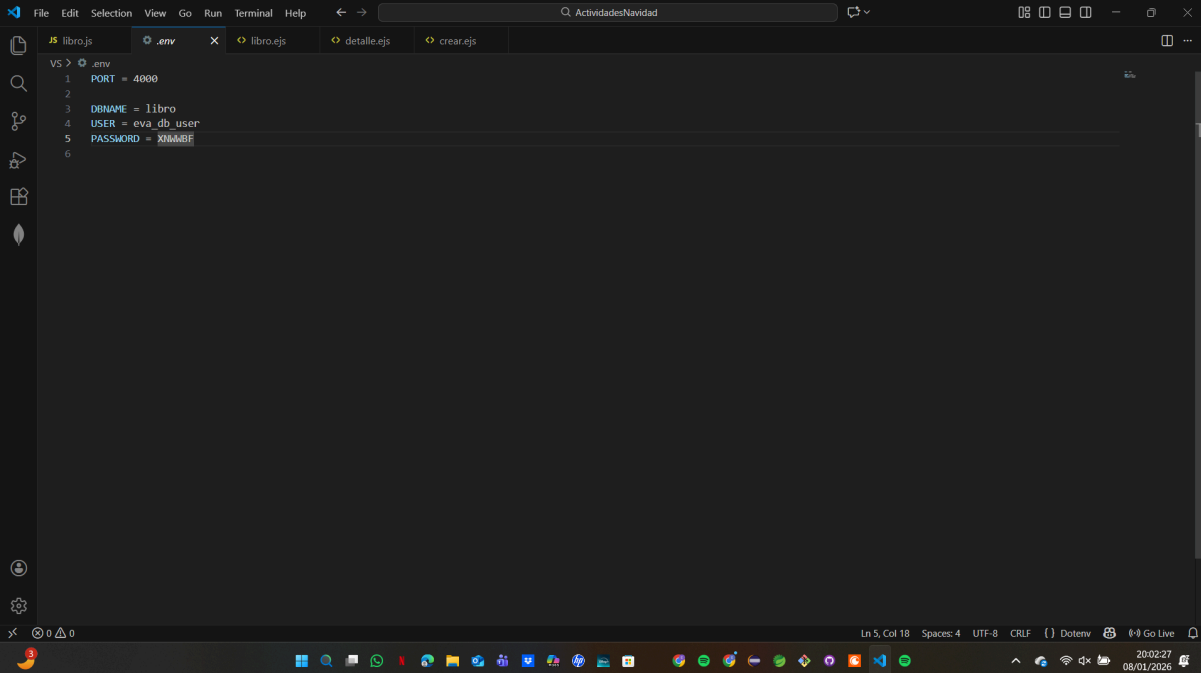
Desarrollo CRUD

En esta actividad, definiremos el CRUD de la entidad más relevante de nuestro proyecto. En mi caso, he escogido la entidad de libro con los atributos isbn, título, autor y número de páginas.

Con esta información y apoyándome en las diapositivas explicadas en clase, he conseguido hacer el crud como se mostrará en las siguientes capturas. Antes de empezar, pondré una captura de la **organización del proyecto** de node y haré hincapié en la importancia de la misma ya que un sólo error de organización puede hacer que nuestro servidor no se levante.



También de gran importancia, el archivo `.env` dónde guardaremos los datos de conexión más importantes como la contraseña o el usuario:



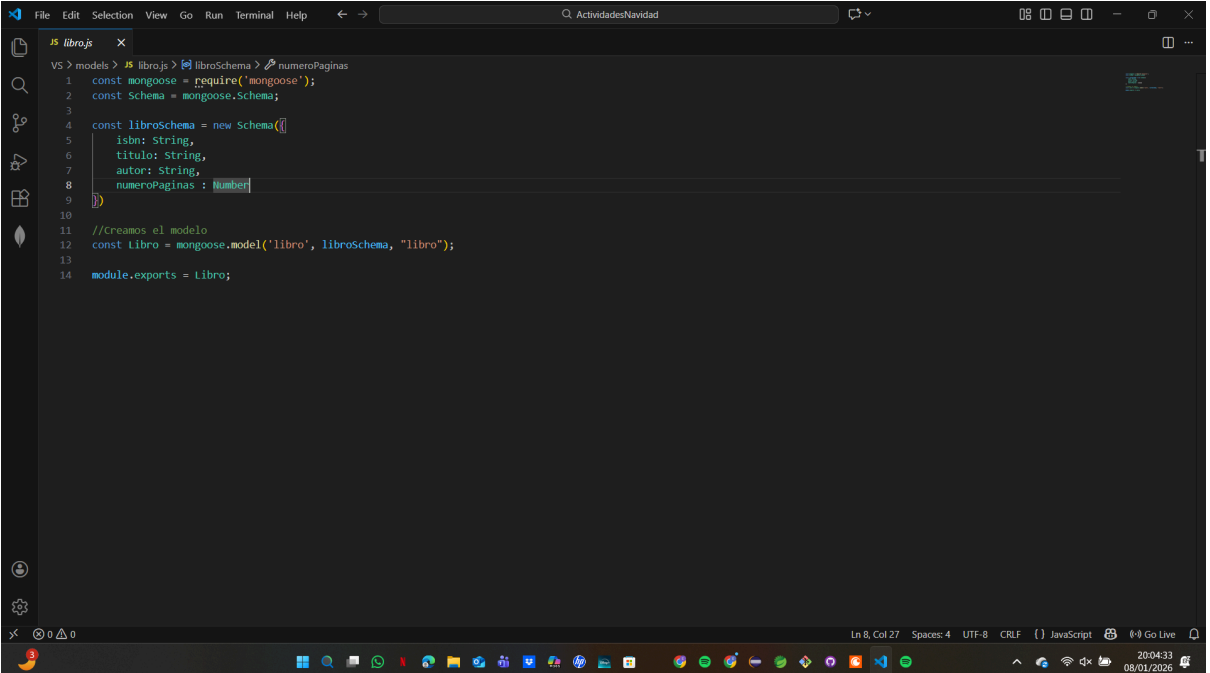
The screenshot shows the Visual Studio Code editor with the `.env` file open. The file contains the following configuration:

```
1 PORT = 4000
2
3 DBNAME = libro
4 USER = eva_db_user
5 PASSWORD = XMMMBF
6
```

The status bar at the bottom indicates the file is at line 5, column 18, with 4 spaces, UTF-8 encoding, and CRLF line endings.

A continuación pondré capturas de pantalla de todas las carpetas y archivos que he utilizado para montar el servidor y conectarlo a la base de datos:

- Models, libro.js

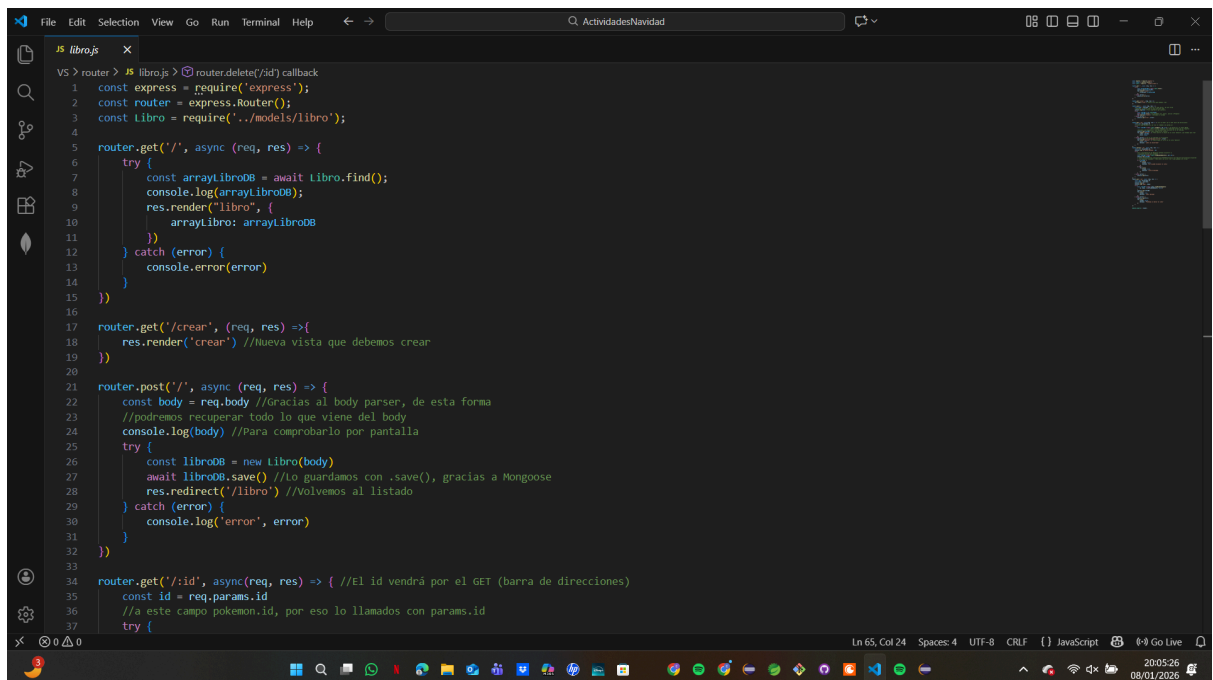


The screenshot shows the Visual Studio Code editor with the `libro.js` file open. The file contains the following code:

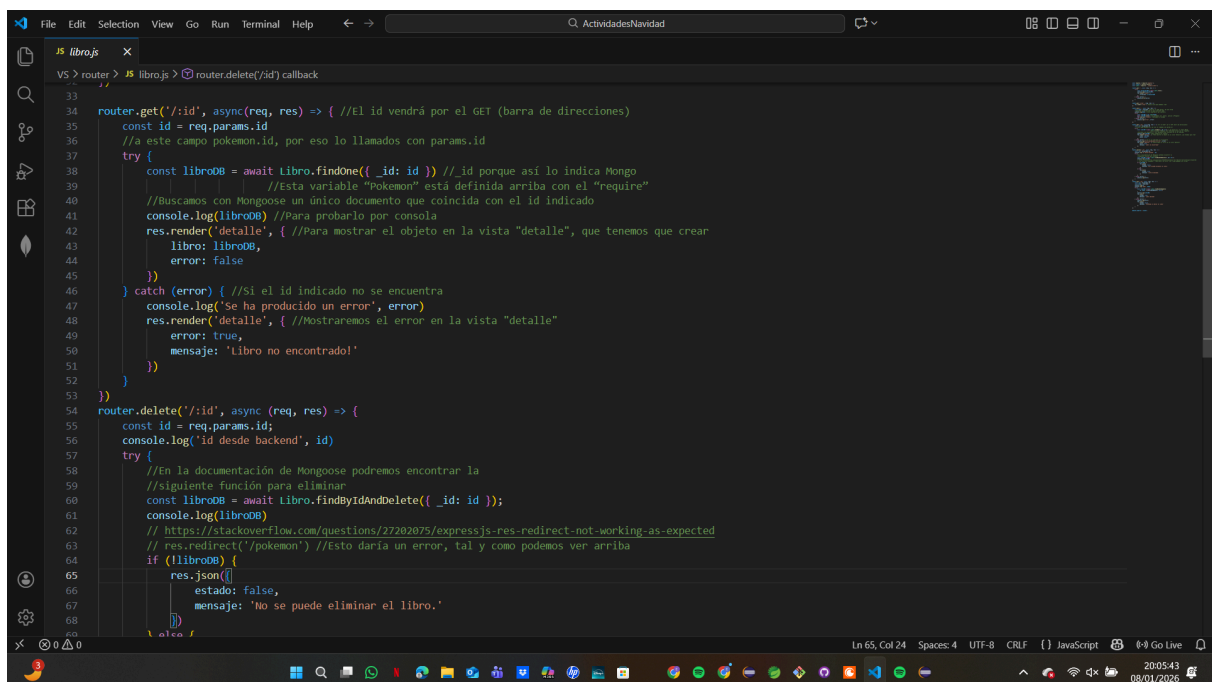
```
1 const mongoose = require('mongoose');
2 const Schema = mongoose.Schema;
3
4 const libroSchema = new Schema({
5   isbn: String,
6   titulo: String,
7   autor: String,
8   numeroPaginas : Number
9 })
10
11 //Creamos el modelo
12 const Libro = mongoose.model('libro', libroSchema, "libro");
13
14 module.exports = Libro;
```

The status bar at the bottom indicates the file is at line 8, column 27, with 4 spaces, UTF-8 encoding, and CRLF line endings.

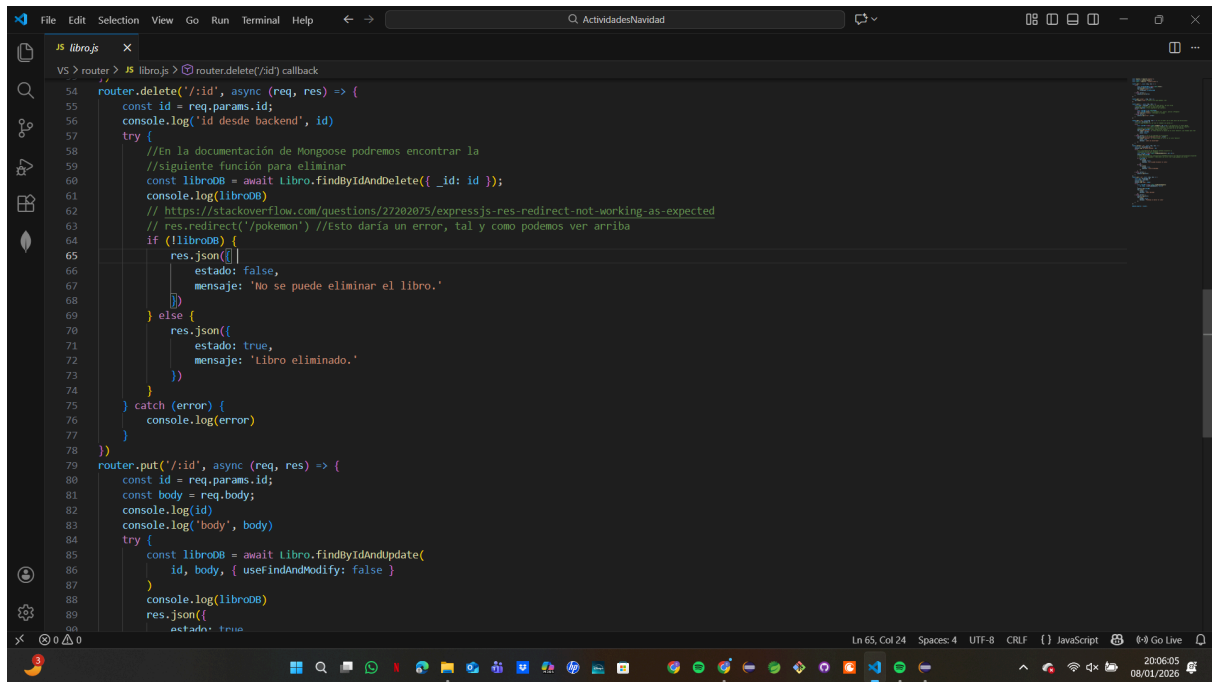
- Router, libro.js, donde tendremos una función para cada una de las acciones que queremos realizar con el libro, es decir, para cada una de las operaciones del CRUD.



```
VS > router > JS libro.js > router.delete('/:id') callback
1 const express = require('express');
2 const router = express.Router();
3 const Libro = require('../models/libro');
4
5 router.get('/', async (req, res) => {
6   try {
7     const arraylibroDB = await Libro.find();
8     console.log(arraylibroDB);
9     res.render('libro', {
10       arrayLibro: arraylibroDB
11     });
12   } catch (error) {
13     console.error(error)
14   }
15 })
16
17 router.get('/crear', (req, res) =>{
18   res.render('crear') //Nueva vista que debemos crear
19 })
20
21 router.post('/', async (req, res) => {
22   const body = req.body //gracias al body parser, de esta forma
23   //podremos recuperar todo lo que viene del body
24   console.log(body) //Para comprobarlo por pantalla
25   try {
26     const libroDB = new Libro(body)
27     await libroDB.save() //Lo guardamos con .save(), gracias a Mongoose
28     res.redirect('/libro') //volvemos al listado
29   } catch (error) {
30     console.log('error', error)
31   }
32 })
33
34 router.get('/:id', async (req, res) => { //El id vendrá por el GET (barra de direcciones)
35   const id = req.params.id
36   //a este campo pokemon.id, por eso lo llamados con params.id
37   try {
```

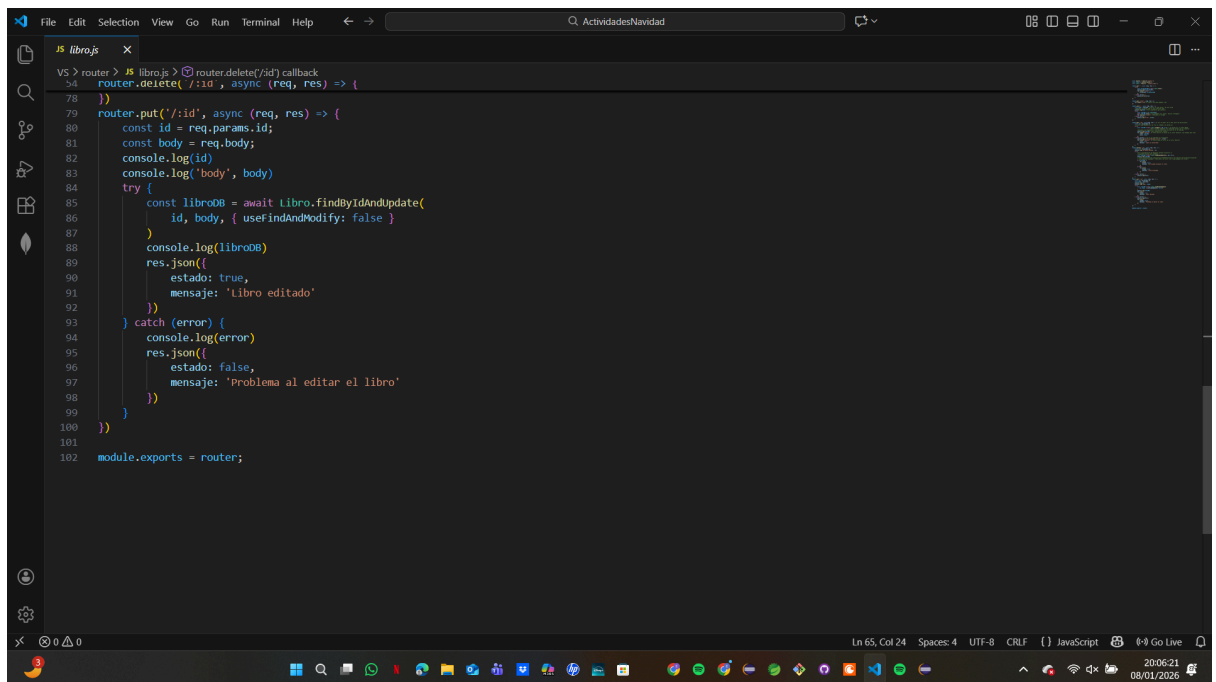


```
38
39   const libroDB = await Libro.findOne({ _id: id }) // id porque así lo indica Mongo
40   //Esta variable "pokemon" está definida arriba con el "require"
41   //Buscamos con Mongoose un único documento que coincida con el id indicado
42   console.log(libroDB) //Para probarlo por consola
43   res.render('detalle', { //Para mostrar el objeto en la vista "detalle", que tenemos que crear
44     libro: libroDB,
45     error: false
46   })
47 } catch (error) { //Si el id indicado no se encuentra
48   console.log("Se ha producido un error", error)
49   res.render('detalle', { //Mostraremos el error en la vista "detalle"
50     error: true,
51     mensaje: 'Libro no encontrado!'
52   })
53 })
54
55 router.delete('/:id', async (req, res) => {
56   const id = req.params.id;
57   console.log('id desde backend', id)
58   try {
59     //En la documentación de Mongoose podremos encontrar la
60     //siguiente función para eliminar
61     const libroDB = await Libro.findByIdAndDelete({ _id: id });
62     console.log(libroDB)
63     // https://stackoverflow.com/questions/27202075/expressjs-res-redirect-not-working-as-expected
64     // res.redirect('/pokemon') //Esto daría un error, tal y como podemos ver arriba
65     if (!libroDB) {
66       res.json({
67         estado: false,
68         mensaje: 'No se puede eliminar el libro.'
69       });
70     }
71   } catch (error) {
72     console.log('error', error)
73   }
74 })
```



The screenshot shows a VS Code editor window with a file named `libro.js`. The code implements a `router.delete('/:id')` endpoint. It uses `req.params.id` to get the book ID, logs it, and then attempts to delete the book using `Libro.findByIdAndDelete`. If the book is found, it returns a JSON response with `estado: false` and `mensaje: 'No se puede eliminar el libro.'`. If the book is not found, it returns a JSON response with `estado: true` and `mensaje: 'Libro eliminado.'`. The code also includes a `catch` block for errors and a `router.put('/:id')` endpoint for updating a book.

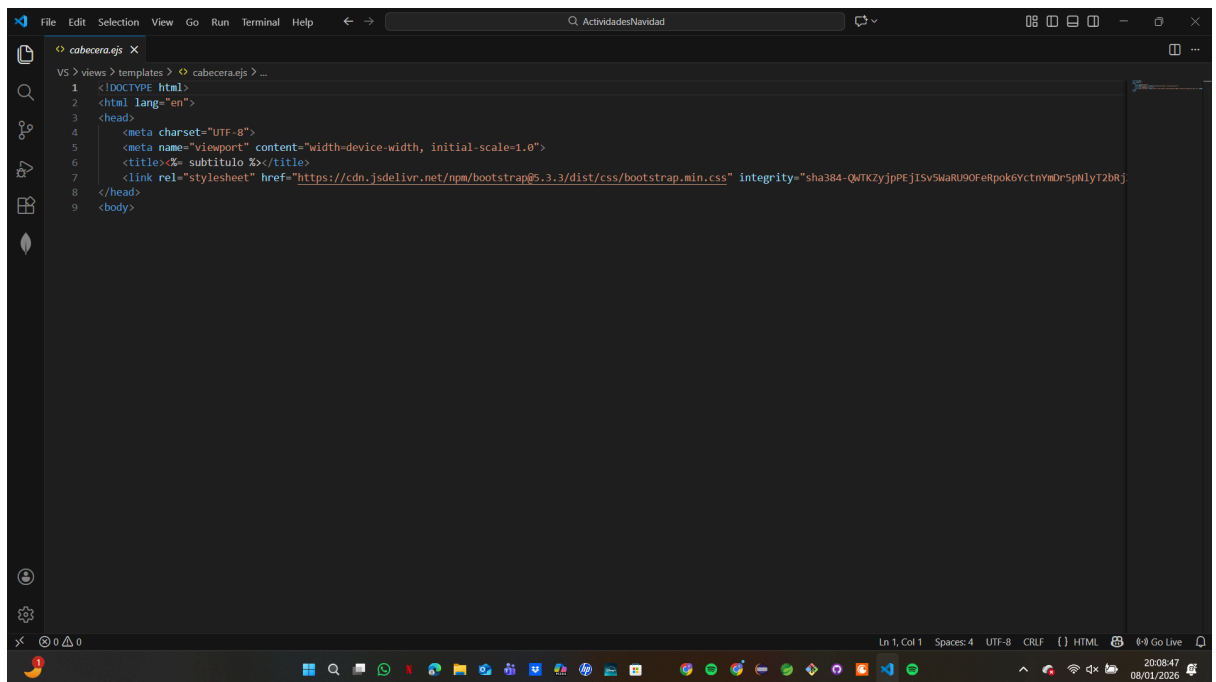
```
VS > router > JS libro.js > router.delete('/:id') callback
54 router.delete('/:id', async (req, res) => {
55   const id = req.params.id;
56   console.log('id desde backend', id)
57   try {
58     //En la documentación de Mongoose podremos encontrar la
59     //siguiente función para eliminar
60     const libroDB = await Libro.findByIdAndDelete({ _id: id });
61     console.log(libroDB)
62     // https://stackoverflow.com/questions/27202075/expressjs-res-redirect-not-working-as-expected
63     // res.redirect('/pokemon') //Esto daría un error, tal y como podemos ver arriba
64     if (!libroDB) {
65       res.json({
66         estado: false,
67         mensaje: 'No se puede eliminar el libro.'
68       })
69     } else {
70       res.json({
71         estado: true,
72         mensaje: 'Libro eliminado.'
73       })
74     }
75   } catch (error) {
76     console.log(error)
77   }
78 }
79 router.put('/:id', async (req, res) => {
80   const id = req.params.id;
81   const body = req.body;
82   console.log(id)
83   console.log('body', body)
84   try {
85     const libroDB = await Libro.findByIdAndUpdate(
86       id, body, { useFindAndModify: false }
87     )
88     console.log(libroDB)
89     res.json({
90       estado: true
91     })
92   } catch (error) {
93     console.log(error)
94     res.json({
95       estado: false,
96       mensaje: 'Problema al editar el libro'
97     })
98   }
99 }
100 }
101 module.exports = router;
```



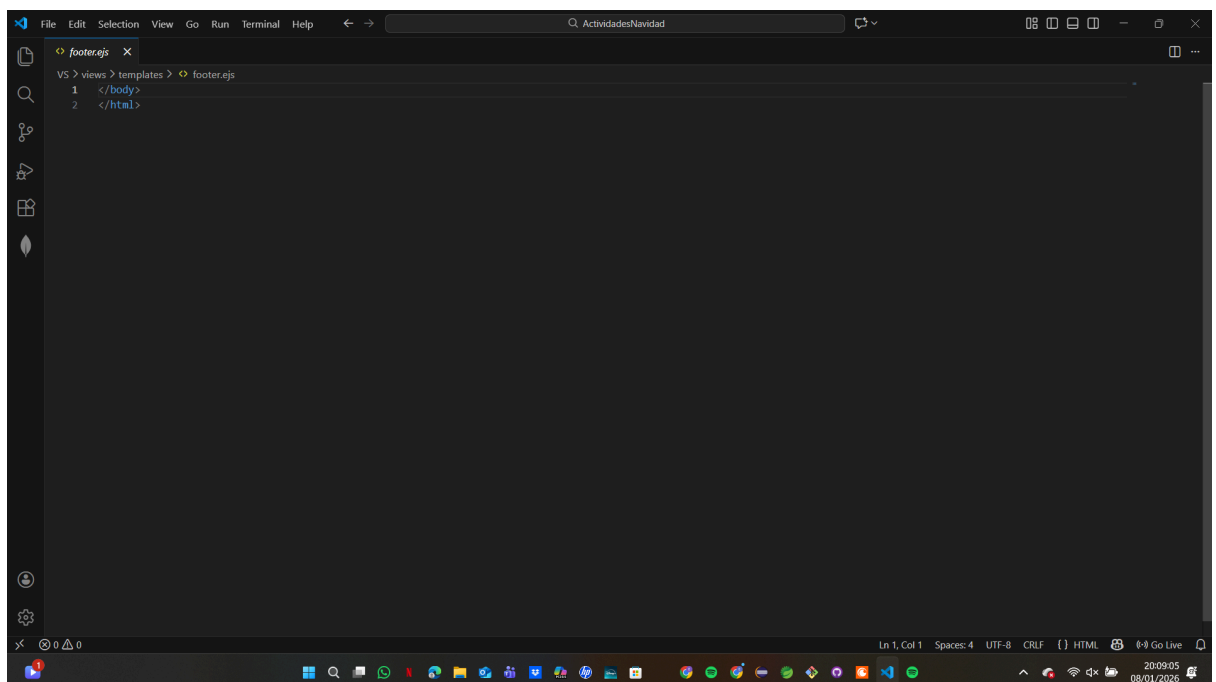
The screenshot shows a VS Code editor window with a file named `libro.js`. The code implements a `router.put('/:id')` endpoint. It uses `req.params.id` to get the book ID, logs it, and then attempts to update the book using `Libro.findByIdAndUpdate`. If the book is found, it returns a JSON response with `estado: true` and `mensaje: 'Libro editado'`. If the book is not found, it returns a JSON response with `estado: false` and `mensaje: 'Problema al editar el libro'`. The code also includes a `catch` block for errors and a `router.delete('/:id')` endpoint for deleting a book.

```
VS > router > JS libro.js > router.delete('/:id') callback
54 router.delete('/:id', async (req, res) => {
55   const id = req.params.id;
56   console.log('id desde backend', id)
57   try {
58     //En la documentación de Mongoose podremos encontrar la
59     //siguiente función para eliminar
60     const libroDB = await Libro.findByIdAndDelete({ _id: id });
61     console.log(libroDB)
62     // https://stackoverflow.com/questions/27202075/expressjs-res-redirect-not-working-as-expected
63     // res.redirect('/pokemon') //Esto daría un error, tal y como podemos ver arriba
64     if (!libroDB) {
65       res.json({
66         estado: false,
67         mensaje: 'No se puede eliminar el libro.'
68       })
69     } else {
70       res.json({
71         estado: true,
72         mensaje: 'Libro eliminado.'
73       })
74     }
75   } catch (error) {
76     console.log(error)
77   }
78 }
79 router.put('/:id', async (req, res) => {
80   const id = req.params.id;
81   const body = req.body;
82   console.log(id)
83   console.log('body', body)
84   try {
85     const libroDB = await Libro.findByIdAndUpdate(
86       id, body, { useFindAndModify: false }
87     )
88     console.log(libroDB)
89     res.json({
90       estado: true,
91       mensaje: 'Libro editado'
92     })
93   } catch (error) {
94     console.log(error)
95     res.json({
96       estado: false,
97       mensaje: 'Problema al editar el libro'
98     })
99 }
100 }
101 module.exports = router;
```

- Views, templates, donde guardaremos cabecera.ejs y footer.ejs, esto hará que nuestros archivos tengan un cierre y un comienzo en común, lo que facilitará el trabajo.

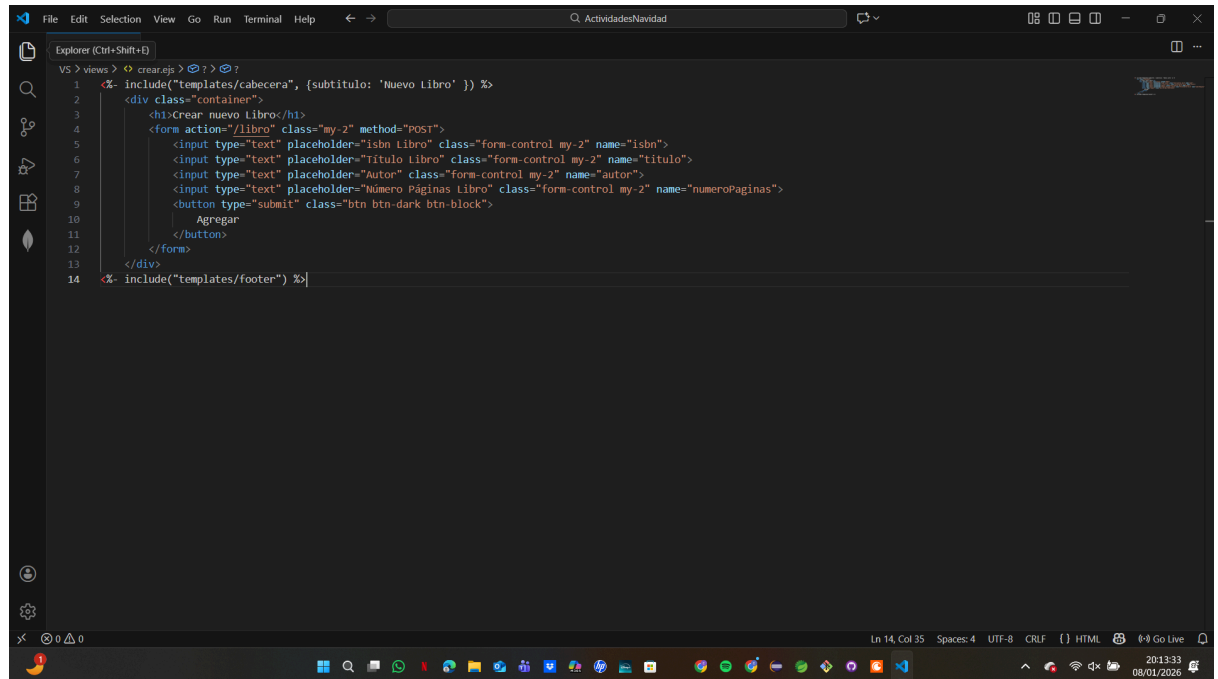


```
VS > views > templates > cabecera.ejs > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title><%= subtitulo %></title>
7   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" integrity="sha384-QMtkZyPpJ4tsF/qA2h6Pv1w8005rRy9fNIeQdMiXmjVyY+Q005eF/MAJjt2B0J" crossorigin="anonymous">
8 </head>
9 <body>
```

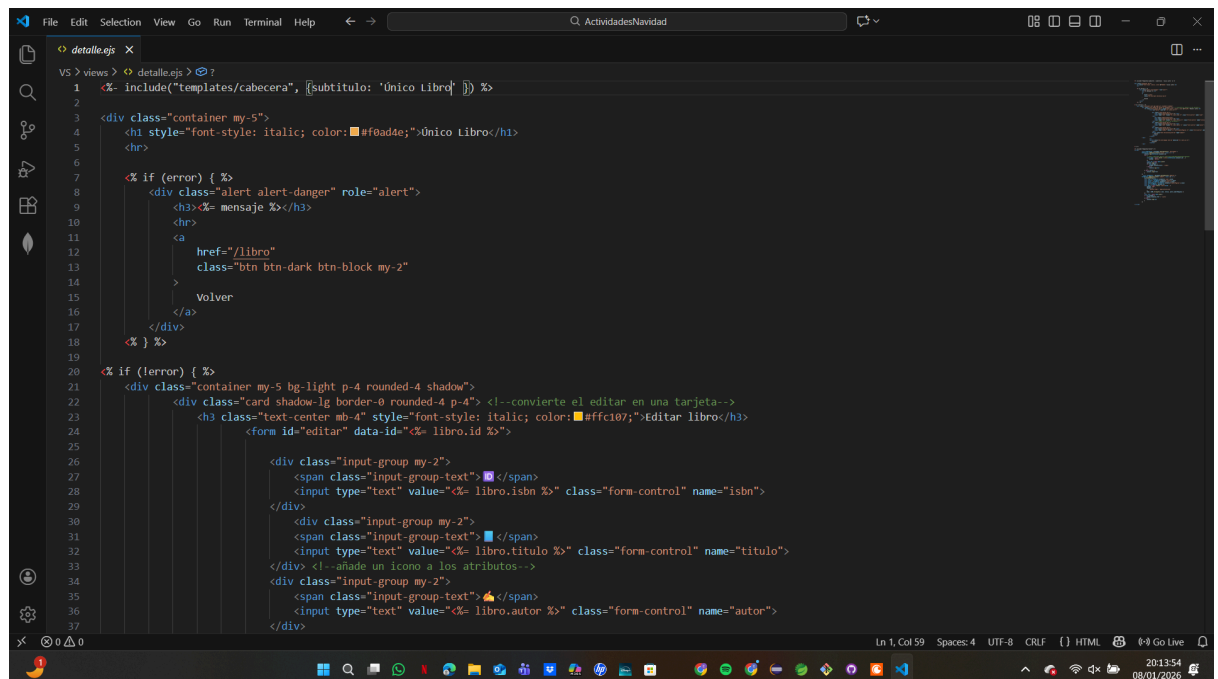


```
VS > views > templates > footer.ejs
1 </body>
2 </html>
```

- Views, encontramos crear.ejs, detalle.ejs y libro.ejs que es lo que le dará la visión a nuestro servidor.



```
VS > views > create.ejs > ?
1 <%= include("templates/cabecera", {subtitulo: 'Nuevo Libro' }) %>
2 <div class="container">
3   <h1>Crear nuevo libro</h1>
4   <form action="/libro" class="my-2" method="POST">
5     <input type="text" placeholder="isbn libro" class="form-control my-2" name="isbn">
6     <input type="text" placeholder="título libro" class="form-control my-2" name="titulo">
7     <input type="text" placeholder="Autor" class="form-control my-2" name="autor">
8     <input type="text" placeholder="Número Páginas Libro" class="form-control my-2" name="numeroPaginas">
9     <button type="submit" class="btn btn-dark btn-block">
10      Agregar
11    </button>
12  </form>
13 </div>
14 <%= include("templates/footer") %>
```



```
VS > views > detalle.ejs > ?
1 <%= include("templates/cabecera", {subtitulo: 'Único Libro' }) %>
2
3 <div class="container my-5">
4   <h1 style="font-style: italic; color: #f0ad4e;">Único Libro</h1>
5   <hr>
6
7   <% if (error) { %>
8     <div class="alert alert-danger" role="alert">
9       <h3><%= mensaje %></h3>
10      <hr>
11      <a
12        href="/libro"
13        class="btn btn-dark btn-block my-2"
14      >
15        volver
16      </a>
17    </div>
18  <% } %>
19
20  <% if (!error) { %>
21    <div class="container my-5 bg-light p-4 rounded-4 shadow">
22      <div class="card shadow-lg border-0 rounded-4 p-4"> <!-- convierte el editar en una tarjeta -->
23        <h3 class="text-center mb-4" style="font-style: italic; color: #ffc107;">Editar libro</h3>
24        <form id="editar" data-id="<%= libro.id %>">
25
26          <div class="input-group my-2">
27            <span class="input-group-text"> <img alt="icon" />
28            <input type="text" value="<%= libro.isbn %>" class="form-control" name="isbn">
29          </div>
30
31          <div class="input-group my-2">
32            <span class="input-group-text"> <img alt="icon" />
33            <input type="text" value="<%= libro.titulo %>" class="form-control" name="titulo">
34          </div> <!-- añade un icono a los atributos -->
35
36          <div class="input-group my-2">
37            <span class="input-group-text"> <img alt="icon" />
38            <input type="text" value="<%= libro.autor %>" class="form-control" name="autor">
39          </div>
40        </form>
41      </div>
42    </div>
43  <% } %>
44 </div>
```



```
File Edit Selection View Go Run Terminal Help Q ActividadesNavidad
detalle.ejs X
VS > views > detalle.ejs > ?
1 <%= include("templates/cabecera", {subtitulo: 'Único Libro' }) %>
3 <div class="container my-5">
7 <% if (error) { %>
18 <% } %>
20 <% if (!error) { %>
22 <div class="card shadow-lg border-0 rounded-4 p-4"> <!-- convierte el editar en una tarjeta -->
23 <h3 class="text-center mb-4" style="font-style: italic; color: #ffc107;">Editar libro</h3>
24 <form id="editar" data-id="<%= libro.id %%">
25
26 <div class="input-group my-2">
27 <span class="input-group-text"> </span>
28 <input type="text" value="<%= libro.isbn %%" class="form-control" name="isbn">
29 </div>
30 <div class="input-group my-2">
31 <span class="input-group-text"> </span>
32 <input type="text" value="<%= libro.titulo %%" class="form-control" name="titulo">
33 </div> <!-- añade un icono a los atributos -->
34 <div class="input-group my-2">
35 <span class="input-group-text"> </span>
36 <input type="text" value="<%= libro.autor %%" class="form-control" name="autor">
37 </div>
38 <div class="input-group my-2">
39 <span class="input-group-text"> </span>
40 <input type="number" value="<%= libro.numeroPaginas %%" class="form-control" name="numeroPaginas">
41 </div>
42 <button class="btn btn-warning btn-sm" type="submit">
43 Editar
44 </button>
45 </form>
46 </div>
47 <hr>
48 <button class="btn btn-danger btn-sm" data-id="<%= libro.id %%">
49 Eliminar
50 </button>
51 </div>
52 <% } %>
53 <% } %>
Ln 1, Col 59 Spaces: 4 UTF-8 CRLF {} HTML Go Live
20:14:26 08/01/2026
```

```
File Edit Selection View Go Run Terminal Help Q ActividadesNavidad
detalle.ejs X
VS > views > detalle.ejs > ?
1 <%= include("templates/cabecera", {subtitulo: 'Único Libro' }) %>
3 <div class="container my-5">
7 <% if (error) { %>
18 <% } %>
20 <% if (!error) { %>
54 <%= include("templates/footer") %>
55 <script>
56
57 const btnEliminar = document.querySelector('.btn-danger')
58 btnEliminar.addEventListener('click', async () => {
59   console.log(btnEliminar.dataset.id)
60   try {
61     // https://developer.mozilla.org/es/docs/Web/API/HTMLElement/dataset
62     const data = await fetch(`/libro/${btnEliminar.dataset.id}`, {
63       method: 'delete'
64     })
65     const res = await data.json()
66     console.log(res)
67     if (res.estado) {
68       window.location.href = '/libro'
69     } else {
70       console.log(res)
71     }
72   } catch (error) {
73     console.log(error)
74   }
75 })
76 const formEditar = document.querySelector('#editar');
77 formEditar.addEventListener('submit', async (e) => {
78   e.preventDefault()
79   const isbn = formEditar.elements['isbn'].value
80   const titulo = formEditar.elements['titulo'].value
81   const autor = formEditar.elements['autor'].value
82   const numeroPaginas = formEditar.elements['numeroPaginas'].value
83   const id = formEditar.dataset.id
84   const data = await fetch(`/libro/${id}`, {
Ln 1, Col 59 Spaces: 4 UTF-8 CRLF {} HTML Go Live
20:14:42 08/01/2026
```

The screenshot shows the Visual Studio Code interface with a file named `detalle.js` open. The code is as follows:

```
<%- include("templates/cabecera", { subtítulo: 'Único Libro' }) %>
<div class="container my-5">
  <% if (error) { %>
    <% } %>
  <% if (!error) { %>
    } catch (error) {
      console.log(error)
    }
  }
}

const formEditor = document.querySelector("#editar");
formEditor.addEventListener("submit", async (e) => {
  e.preventDefault();
  const isbn = formEditor.elements["isbn"].value;
  const título = formEditor.elements["título"].value;
  const autor = formEditor.elements["autor"].value;
  const númeroPaginas = formEditor.elements["númeroPaginas"].value;
  const id = formEditor.dataset.id;
  const data = await fetch(`/libro/${id}`, {
    method: "PUT",
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ isbn, título, autor, númeroPaginas })
  });
  const res = await data.json();
  if (res.estado) {
    window.location.href = '/libro'
  } else {
    console.log(res);
  }
})
</script>
```

The status bar at the bottom indicates the cursor is at Line 1, Column 59, with 4 spaces, UTF-8 encoding, and CR/LF line endings.

The screenshot shows a web browser window with a dark theme. The address bar shows the URL "http://localhost:3000/". The page content is as follows:

```
<!-- Título centrado -->
<h1 class="text-center" style="font-style: italic; color: #f0ad4e;">CRUD Ready</h1>

<!-- Botón Crear Libro -->
<div class="text-center mb-4">
  <a class="btn btn-warning" href="/libro/crear" style="font-weight:bold;">
    Crear nuevo Libro
  </a>
</div>

<!-- Tarjeta para la tabla -->
<div class="card shadow-lg rounded-4 p-4">
  <table class="table table-hover align-middle">
    <thead class="table-dark rounded-top">
      <tr>
        <th scope="col">#</th>
        <th scope="col">ISBN</th>
        <th scope="col">Titulo</th>
        <th scope="col">Autor</th>
        <th scope="col">Número páginas</th>
        <th scope="col">Acciones</th>
      </tr>
    </thead>
    <tbody>
      <% if (arrayLibro && arrayLibro.length > 0) { %>
        <% arrayLibro.forEach(libro => { %>
          <tr>
            <th scope="row">%= libro.id %</th>
            <td>%= libro.isbn %</td>
            <td>%= libro.titulo %</td>
            <td>%= libro.autor %</td>
            <td>%= libro.numeroPaginas %</td>
          </tr>
        </td>
      </td>
    </td>
  </td>

```

```

1  <%- include("templates/cabecera", {subtitulo: 'Librería Digital : Ready'}) %>
2  <div class="container my-5">
3    <div class="card shadow-lg rounded-4 p-4">
4      <table class="table table-hover align-middle">
5        <thead class="table-dark rounded-top">
6          <tr>
7            <th>ID</th>
8            <th>ISBN</th>
9            <th>Título</th>
10           <th>Autor</th>
11           <th>Número páginas</th>
12           <th>Acciones</th>
13         </thead>
14         <tbody>
15           <% if (arrayLibro && arrayLibro.length > 0) { %>
16             <% arrayLibro.forEach(libro => { %>
17               <tr>
18                 <th scope="row"><%= libro.id %></th>
19                 <td><%= libro.isbn %></td>
20                 <td><%= libro.titulo %></td>
21                 <td><%= libro.autor %></td>
22                 <td><%= libro.numeroPaginas %></td>
23                 <td>
24                   <a class="btn btn-warning" href="/libro/<%= libro.id %>">
25                     Editar/Eliminar
26                   </a>
27                 </td>
28               </tr>
29             <% } %>
30           <% } else { %>
31             <td colspan="6" class="text-center">No hay libros registrados.</td>
32           <% } %>
33         </tbody>
34       </table>
35     </div>
36   </div>
37   <%- include("templates/footer") %>

```

Finalizando, inicializamos el servidor que hemos montado anteriormente:

```

PS C:\Users\evape\OneDrive\Escritorio\segundo_DAM\op\ActividadesNavidad\VS> node .\01-express.js
[dotenv@17.2.3] injecting env (4) from .env -- tip: backup and recover secrets: https://dotenvx.com/ops
Example app listening on port 4000
Base de datos conectada

```

Si todo ha salido como esperábamos, la terminal tendría que verse de esta manera.

Por último, la pantalla quedaría de la siguiente manera:

